

Chuleta de Complejidad: Problemas de Optimización NPO

Cubrimiento Mínimo por Vértices — APX-completo

Enunciado

Dado un grafo no dirigido $G = (V, E)$, encontrar el subconjunto $C \subseteq V$ de tamaño mínimo tal que toda arista tenga al menos un extremo en C . Coste de una solución: su número de vértices. **Clase:** APX-completo. No se puede aproximar con $\delta = 1,1659$ (si $P \neq NP$).

Algoritmo greedy (grado máximo) — NO es δ -aproximado para ningún δ

$C = \emptyset$. Mientras haya aristas: elegir el nodo de grado máximo, añadirlo a C , borrar ese nodo y todas sus aristas.

Contraejemplo: Se construye un grafo tripartito con nodos centrales $b_1, \dots, b_n$, nodos a la izquierda $a_1, \dots, a_k$ y nodos a la derecha $c_1, \dots, c_l$. Los nodos de la derecha se conectan con grupos decrecientes de nodos centrales (de tamaño $n, n/2, n/3, \dots$). El greedy puede elegir todos los nodos de la derecha (muchos), cuando bastaría con los n centrales. La razón alcanza $\frac{n+l}{n} \approx \log(n)$ para n nodos centrales. Por tanto el error puede ser de orden $\log(n)$.

Algoritmo 2-aproximado — demostración

$C = \emptyset$. Mientras haya aristas en G : elegir una arista cualquiera, añadir sus dos extremos a C , borrar ambos nodos y todas sus aristas.

Corrección: Toda arista queda cubierta: o bien fue la arista elegida (ambos extremos en C), o bien al menos uno de sus extremos fue eliminado en algún paso previo y, por tanto, ya estaba en C .

Razón 2: Sea M el conjunto de aristas elegidas por el algoritmo. Como los dos extremos de cada arista elegida se borran inmediatamente, las aristas de M no comparten vértices entre sí, luego $|C| = 2|M|$. Para cubrir las aristas de M , toda solución óptima necesita al menos un extremo de cada una (son disjuntas), así que $\text{OPT} \geq |M|$. Por tanto:

$$\frac{|C|}{\text{OPT}} = \frac{2|M|}{\text{OPT}} \leq \frac{2|M|}{|M|} = 2$$

Es el mejor algoritmo conocido para grafos generales.

Corte Máximo — APX-completo

Enunciado

Dado un grafo no dirigido $G = (V, E)$ sin lazos, partir V en dos conjuntos S y $V \setminus S$ de forma que el número de aristas entre ambos sea máximo. El problema de decisión es NP-completo.

Algoritmo greedy 2-aproximado — demostración

Para cada vértice v y conjunto A , sea $\text{arc}(v, A)$ el número de aristas entre v y nodos de A .

Algoritmo: Comenzar con un S arbitrario. Mientras S cambie: para cada vértice v : si $v \in S$ y $\text{arc}(v, V \setminus S) < \text{arc}(v, S)$, eliminar v de S ; si $v \notin S$ y $\text{arc}(v, V \setminus S) > \text{arc}(v, S)$, añadir v a S .

Terminación: Cada movimiento incrementa estrictamente el corte. Como $|E|$ es finito, el algoritmo termina en a lo sumo $|E|$ pasos, y cada paso es polinómico.

Razón 2: Sea S^* el corte óptimo. Partimos V en cuatro conjuntos: $V_1 = S \cap S^*$, $V_2 = S \setminus S^*$, $V_3 = S^* \setminus S$, $V_4 = (V \setminus S) \setminus S^*$. Sea e_{ij} el número de aristas entre V_i y V_j . De la condición de estabilidad del greedy: para cada nodo, las aristas hacia su propio lado son $\leq$ las aristas hacia el lado opuesto. Sumando sobre cada V_k :

- $V_1 \subset S$: para cada $x_1 \in V_1$, $\text{arc}(x_1, V_1) + \text{arc}(x_1, V_2) \leq \text{arc}(x_1, V_3) + \text{arc}(x_1, V_4)$. Sumando en V_1 : $2e_{11} + e_{12} \leq e_{13} + e_{14}$, luego $e_{12} \leq e_{13} + e_{14}$.
- $V_2 \subset S$: análogamente, $e_{12} \leq e_{23} + e_{24}$.
- $V_3 \subset V \setminus S$: $e_{34} \leq e_{23} + e_{13}$.
- $V_4 \subset V \setminus S$: $e_{34} \leq e_{14} + e_{24}$.

Sumando las cuatro desigualdades y dividiendo por 2:

$$e_{12} + e_{34} \leq e_{13} + e_{14} + e_{23} + e_{24}$$

También trivialmente $e_{14} + e_{23} \leq e_{13} + e_{14} + e_{23} + e_{24}$. Sumando ambas:

$$\underbrace{e_{12} + e_{34} + e_{14} + e_{23}}_{\text{OPT}} \leq 2 \cdot \underbrace{(e_{13} + e_{14} + e_{23} + e_{24})}_{\text{APROX}}$$

donde OPT son las aristas del corte óptimo (entre $\{V_1, V_3\}$ y $\{V_2, V_4\}$) y APROX las del heurístico (entre $\{V_1, V_2\}$ y $\{V_3, V_4\}$). Así, $\frac{\text{OPT}}{\text{APROX}} \leq 2$.

Satisfacción Máxima: K-GMAXSAT — APX

Enunciado

Dadas n variables $p_1, \dots, p_n$ y m fórmulas booleanas $\Phi = \{\phi_1, \dots, \phi_m\}$ con a lo más K variables cada una, encontrar una asignación de valores de verdad que maximice el número de fórmulas satisfechas.

Algoritmo aproximado — demostración

Esperanza al azar: Para una asignación aleatoria uniforme, sea t_i el número de asignaciones a las $\leq k$ variables de ϕ_i que la hacen cierta. Entonces $P(\phi_i) = t_i/2^k$. El número esperado de fórmulas satisfechas es, por linealidad de la esperanza:

$$E(\Phi) = \sum_{i=1}^m P(\phi_i)$$

Para una cláusula $l_1 \vee \dots \vee l_k$ con k literales distintos: $P = (2^k - 1)/2^k = 1 - 2^{-k}$.

Método de esperanzas condicionales: Fijamos las variables una a una. Para la variable x_1 :

$$E(\Phi) = \frac{1}{2} (E(\Phi[x_1 = V]) + E(\Phi[x_1 = F]))$$

donde $\Phi[x_1 = T]$ es el conjunto de fórmulas obtenido sustituyendo x_1 por T y simplificando. Elegimos el valor que da mayor esperanza condicional; así la esperanza nunca decrece. Repetimos para cada variable sucesivamente.

Al terminar (todas las variables fijadas), la esperanza es el número exacto de fórmulas satisfechas, que es $\geq E(\Phi)$ original.

Razón de aproximación: El óptimo es $\leq \#\{\phi_i : P(\phi_i) > 0\}$ (solo pueden satisfacerse las fórmulas satisfacibles). Además, $\text{APROX} \geq E(\Phi) = \sum P(\phi_i)$. Luego:

$$\frac{\text{OPT}}{\text{APROX}} \leq \frac{\#\{\phi_i : P(\phi_i) > 0\}}{\sum P(\phi_i)} \leq \frac{1}{\min\{P(\phi_i) : P(\phi_i) > 0\}}$$

- Para fórmulas generales con $\leq k$ variables: $\delta = 2^k$ (pues $P(\phi_i) \geq 1/2^k$).
- Para cláusulas con k literales distintos: $P = 1 - 2^{-k}$, luego $\delta = \frac{1}{1-2^{-k}}$.
- Para cláusulas generales (sin restricción de tamaño): $\delta = 2$.

Viajante de Comercio — NPO-completo

Enunciado

Dado un conjunto de ciudades y distancias $d(i, j)$ entre ellas, encontrar el circuito que visita todas las ciudades exactamente una vez con coste total mínimo.

Teorema: No tiene algoritmo con umbral δ finito (si $P \neq NP$) — demostración

Supongamos que existe un algoritmo δ -aproximado polinómico para algún $\delta < \infty$. Construimos un algoritmo polinómico para el Circuito Hamiltoniano (NP-completo), lo que contradice $P \neq NP$.

Dado un grafo $G = (V, E)$, construimos un problema del viajante con los mismos nodos y distancias:

$$d(i, j) = \begin{cases} 1 & \text{si } (i, j) \in E \\ |V| \cdot \delta & \text{si } (i, j) \notin E \end{cases}$$

Aplicamos el algoritmo δ -aproximado:

- **Si G tiene circuito hamiltoniano:** $\text{OPT} = |V|$ (todas las aristas de coste 1). El algoritmo devuelve un circuito de coste $\leq \delta \cdot |V|$. Pero cualquier circuito que use al menos una arista fuera de E tiene coste $\geq |V| \cdot \delta + (|V| - 1) > |V| \cdot \delta$. Luego un coste $\leq |V| \cdot \delta$ implica que el circuito usa solo aristas de E : **es un circuito hamiltoniano en G** , y lo hemos encontrado.

- **Si G no tiene circuito hamiltoniano:** $\text{OPT} > |V|$ (el óptimo usa al menos una arista de coste $|V| \cdot \delta$). Como $\frac{\text{Coste}}{\text{OPT}} \leq \delta$, deducimos $\text{OPT} \geq \frac{\text{Coste}}{\delta} > |V|$. Así, si el circuito devuelto tiene coste $> |V| \cdot \delta$, sabemos que G no tiene circuito hamiltoniano.

Este procedimiento decide el Circuito Hamiltoniano en tiempo polinómico, contradicción con $P \neq NP$. Luego no puede existir tal algoritmo δ -aproximado para ningún δ finito.

NP-dureza de la versión de optimización — demostración

El problema de decisión del viajante de comercio es NP-completo. Como resolver la versión de optimización permite resolver la de decisión (basta comparar el coste óptimo con la cota B), tenemos que VC-decisión $\leq_T$ VC-optimización, y por tanto la versión de optimización es NP-dura.

Recíprocamente, VC-optimización $\leq_T$ VC-decisión mediante la siguiente reducción Turing:

- **Calcular el coste óptimo K :** Sea R el valor de distancia más grande multiplicado por el número de ciudades (cota superior trivial). Mediante búsqueda binaria en $[0, R]$ con sucesivas llamadas al problema de decisión, se obtiene K en $O(\log R)$ llamadas.
- **Reconstruir el circuito óptimo:** Para cada par de ciudades (i, j) con coste c , incrementar su distancia a $c + 1$ y preguntar si el óptimo sigue siendo $\leq K$. Si la respuesta es sí, ese arco no es necesario en el óptimo: dejar el coste en $c + 1$ y continuar. Si la respuesta es no, ese arco sí se usa: restaurar el coste a c y continuar. Al terminar, los arcos no incrementados forman un circuito óptimo.

El incremento se deja en $c + 1$ (no se restaura) para evitar ambigüedad cuando hay múltiples circuitos óptimos: si se restaurase, ningún arco individual sería necesario y no se identificaría ninguno.

Problema de la Mochila — FPTAS, pseudo-polinómico

Enunciado

Dados pesos $w_1, \dots, w_n$, valores $v_1, \dots, v_n$ y peso límite W , encontrar $S \subseteq \{1, \dots, n\}$ tal que $\sum_{i \in S} w_i \leq W$ y $\sum_{i \in S} v_i$ sea máximo.

Algoritmo pseudo-polinómico

Sea $V = \max\{v_1, \dots, v_n\}$. Definimos $W(i, v)$ como el mínimo peso que se puede alcanzar eligiendo ítems entre los i primeros con valor exactamente v :

$$W(i+1, v) = \min \left\{ \underbrace{W(i, v)}_{\text{no elijo } i+1}, \underbrace{W(i, v - v_{i+1}) + w_{i+1}}_{\text{elijo } i+1} \right\}$$

para $i = 0, \dots, n-1$ y $0 \leq v \leq nV$. El óptimo se obtiene recorriendo $k = nV$ hasta $k = 0$: el primer k con $W(n, k) \leq W$ es el valor óptimo. **Complejidad:** $O(n^2V)$, pseudo-polinómico (depende de la magnitud de V , no solo de su tamaño en bits).

Algoritmo FPTAS — demostración

Idea: Truncar los b bits menos significativos de los valores. Dado $I = (w_1, \dots, w_n, W, v_1, \dots, v_n)$, definimos $I' = (w_1, \dots, w_n, W, v'_1, \dots, v'_n)$ con $v'_i = 2^b \lfloor v_i / 2^b \rfloor$. Aplicamos el algoritmo pseudo-polinómico a I' (sin los ceros finales, es decir, trabajando con $\lfloor v_i / 2^b \rfloor$), que tiene complejidad $O(n^2V/2^b)$.

Demostración de que es δ -aproximado: Sea S la solución óptima de I y S' la que obtiene el algoritmo. Tenemos:

$$\sum_{i \in S'} v_i \geq \sum_{i \in S'} v'_i \geq \sum_{i \in S} v'_i \geq \sum_{i \in S} (v_i - 2^b) = \left(\sum_{i \in S} v_i \right) - n \cdot 2^b$$

La primera desigualdad vale porque $v'_i \leq v_i$. La segunda porque S' es óptimo del problema truncado (y S también es factible para él). La tercera porque $v'_i \geq v_i - 2^b$. Así:

$$V(S) \leq V(S') + n \cdot 2^b$$

Si $V(S') \geq V$ (fácil de garantizar si al menos un ítem cabe):

$$\frac{V(S)}{V(S')} \leq 1 + \frac{n \cdot 2^b}{V(S')} \leq 1 + \frac{n \cdot 2^b}{V}$$

Eligiendo $b = \lfloor \log_2((\delta - 1)V/n) \rfloor$ obtenemos $\frac{n \cdot 2^b}{V} \leq \delta - 1$, luego $\frac{V(S)}{V(S')} \leq \delta$.

Complejidad: $O\left(\frac{n^2V}{(\delta-1)V/n}\right) = O\left(\frac{n^3}{\delta-1}\right)$, polinómico en n y en $\frac{1}{\delta-1}$. Es un esquema de aproximación polinómico total (FPTAS).

Conjunto Independiente Máximo — no en APX (si $P \neq NP$)

Enunciado

Dado un grafo $G = (V, E)$, encontrar el subconjunto $A \subseteq V$ de tamaño máximo tal que no haya aristas entre nodos de A . No está en APX (si $P \neq NP$).

Teorema — demostración

Si existe un algoritmo δ_0 -aproximado polinómico con $\delta_0 < \infty$, entonces existe un esquema de aproximación polinómico.

Construcción del grafo producto G^2 : Dado $G = (V, E)$, definimos $G^2 = (V \times V, E')$ donde

$$((u, u'), (v, v')) \in E' \iff ((u, v) \in E) \vee (u = v \wedge (u', v') \in E)$$

Resultado intermedio: G tiene un conjunto independiente de tamaño k si y solo si G^2 tiene uno de tamaño k^2 .

- $(\Rightarrow)$ Si S es independiente en G de tamaño k , entonces $S \times S$ es independiente en G^2 de tamaño k^2 : para dos nodos distintos $(u, u'), (v, v') \in S \times S$, o bien $u \neq v$ y $(u, v) \notin E$ (porque S es independiente), o bien $u = v$ y $u' \neq v'$ con $(u', v') \notin E$.
- $(\Leftarrow)$ Si H es independiente en G^2 de tamaño k^2 , consideramos las proyecciones $H_1 = \{u \in V : \exists v, (u, v) \in H\}$ y para cada $u \in V$, $H_u = \{v \in V : (u, v) \in H\}$. Todos son conjuntos independientes de G . Como $|H| = k^2$, al menos uno de estos conjuntos tiene tamaño $\geq k$.

Aplicación: Supongamos un algoritmo δ_0 -aproximado de orden $O(n^l)$. Lo aplicamos a G^2 (de tamaño $|V|^2$), con complejidad $O(n^{2l})$. Si el óptimo en G es k , el óptimo en G^2 es k^2 , y el algoritmo encuentra H con $k^2/\text{tam}(H) \leq \delta_0$. Eligiendo H' como el mayor entre H_1 y los H_u , obtenemos $\text{tam}(H') \geq \sqrt{\text{tam}(H)}$, luego:

$$\frac{k}{\text{tam}(H')} \leq \frac{k}{\sqrt{\text{tam}(H)}} \leq \sqrt{\delta_0} = \delta_1$$

Iteración: Repitiendo r veces, obtenemos un algoritmo δ_r -aproximado con $\delta_r = \sqrt[r]{\delta_0}$, de orden $O(n^{2^r \cdot l})$. Como $\delta_r \rightarrow 1$ cuando $r \rightarrow \infty$, para cualquier $\delta > 1$ existe r suficiente. El algoritmo es polinómico (de grado $2^r \cdot l$, que depende de δ pero no de n). Esto constituye un esquema de aproximación polinómico (PTAS).

Problemas completos (resumen)

NPO-completos

Máxima y mínima satisfacción ponderada, máxima y mínima programación lineal $\{0, 1\}$, viajante de comercio.

APX-completos

Satisfacción máxima de cláusulas de longitud 3 (MAX-3SAT), de longitud 2 (MAX-2SAT), corte máximo, viajante de comercio con desigualdad triangular, mínimo cubrimiento por vértices, mínimo conjunto de ruptura de ciclos en grafos dirigidos.